



Workpath SDK Print Sample Overview

Product Group: HP Workpath SDK

Version: 1.6.3

Revised: November 11, 2025

HP Inc.
11311 Chinden Blvd.
Boise, ID 83714 USA

HP Confidential

Table of Contents

1	Introduction	3
1.1	Document conventions	3
2	Print Sample	4
2.1	Print Sample on a device	4
3	File location for print on a device.....	5
4	Initializing SDK.....	6
5	Integrating with print functions	8
5.1	Configuring Print Attributes.....	8
5.2	Retrieving Capabilities	10
5.2.1	Needs of Capabilities	10
5.2.2	Reading Capabilities.....	11
5.2.3	Getting Printer Status and Tray information	12
5.2.4	Getting defaults.....	14
5.2.5	Error handling	15
5.3	Print job submission.....	16
5.3.1	Batch Jobs	16
5.3.2	Creating Job Attributes for Print.....	16
5.3.3	Background/ Foreground Jobs	21
5.3.4	Submitting a job.....	21
5.4	Monitoring Job.....	22
6	Legal notice	24
7	Support.....	25

1 Introduction

Print Sample provides use case scenario how to print a file from different sources such as local storage, remote http server and USB.

1.1 Document conventions

The following conventions are used throughout this document to define, describe, and discuss the API: function prototypes, data structure definitions, enumerated constants.

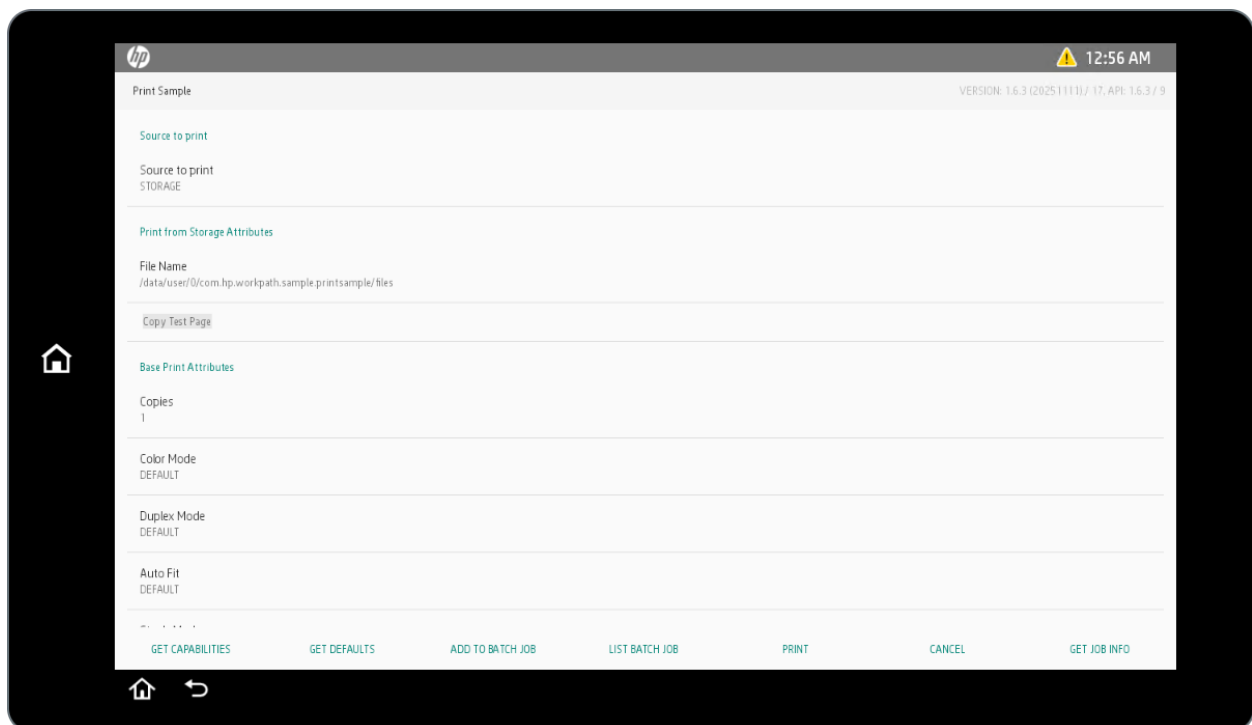
- **Name** – Bold signifies item names such as button names and menu items.
- <name> – Angle brackets mark URLs.
- {value} – Curly braces represent values to be replaced.
- Identifiers for defined constants use all UPPERCASE letters.
- *Name* – Italicized type refers to formal function parameters, data structure types and fields, and other defined or enumerated types in the text body (for example, *streamtype*).
- Numbers are in decimal format unless otherwise noted.
- Name - Monospaced type is used for code fragments, function prototypes and data type definitions.
- Function names are usually accompanied by parentheses (for example, `function()`, `function()`).

2 Print Sample

Print Sample explains how to:

- Interact with Print Functions
 - Retrieve the capabilities using **PrinterService**.
 - Builds print job with printAttributes and submits it.
- Interact with Job Manager
 - Call **JobService** to monitor the current job
 - Handle **JobService.AbstractJobletObserver** callbacks

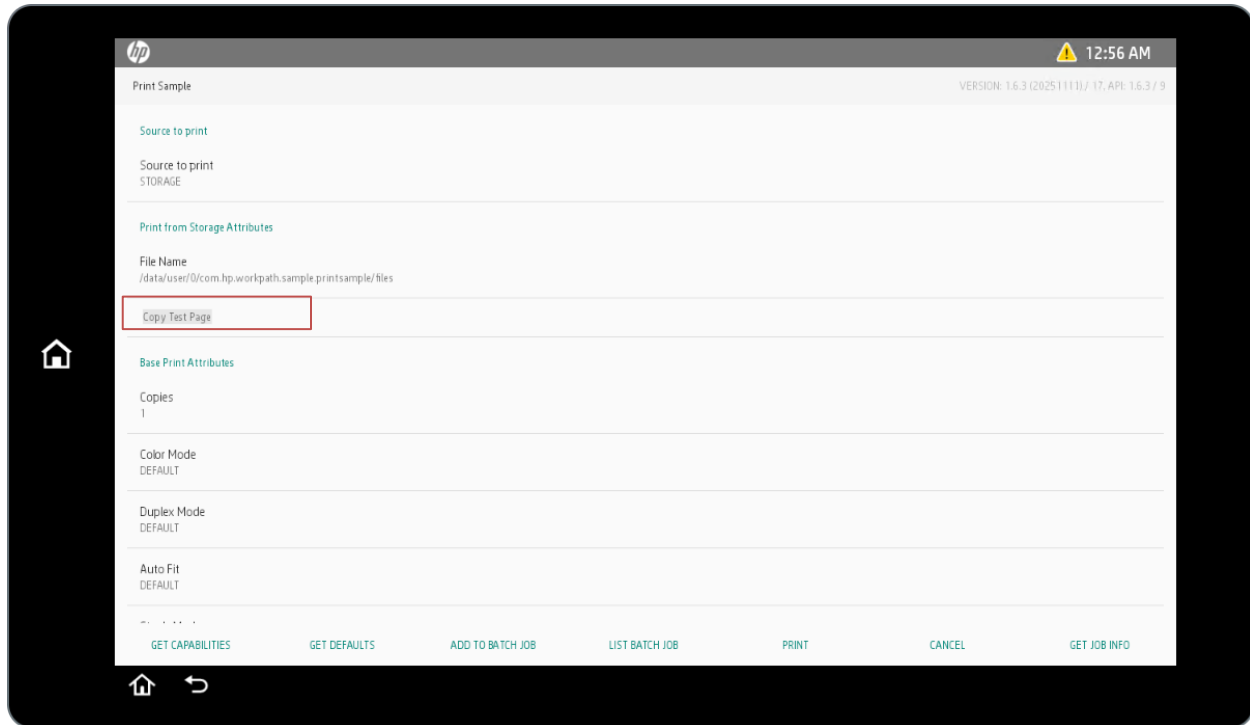
2.1 Print Sample on a device



A main activity of Print Sample on a device

3 File location for print on a device

To test print job, put your file into "{app-private folder}/files" on a device. {app-private folder} can find with calling {applicationContext}.getFilesDir().getPath(). PrintSample provides the menu of "Copy Test Page" to create test files for print from local files.



4 Initializing SDK

Application requires initializing the SDK services when starts with an activity for accessing the SDK services. The `com.hp.workpath.api.Workpath` includes `initialize()` method for this.

The sample shows how to initialize SDK.

1. Call `initialize()` method of Workpath SDK on resume event, and clear it on pause event. It is for calling initialization when an activity is switched. Refer to `MainActivity.java`.

```

1. @Override
2. protected void onResume() {
3.     super.onResume();
4.
5.     // Register JobObserver to receive job state callbacks
6.     mJobObserver.register(getApplicationContext());
7.
8.     mContainer.setEnabled(false);
9.     mInitializationTask = new InitializationTask(this);
10.    mInitializationTask.execute();
11.
12.    mFragment = new PrintConfigureFragment();
13.    getSupportFragmentManager().beginTransaction()
14.        .replace(R.id.dataContainer, mFragment)
15.        .commit();
16. }
17.
18. @Override
19. protected void onPause() {
20.     super.onPause();
21.
22.     mInitializationTask.cancel(true);
23.     mInitializationTask = null;
24.
25.     mJobObserver.unregister(getApplicationContext());
26.
27.     if (mAlertDialog != null) {
28.         mAlertDialog.dismiss();
29.         mAlertDialog = null;
30.     }
31.
32.     if (mSnackBar != null) {
33.         mSnackBar.dismiss();
34.         mSnackBar = null;
35.     }
36. }

```

Initialization call in Activity Resume

Make a task and call `initialize()` method with catching exceptions. If Workpath SDK is not initialized, `sdk exception` or `security exception` will be thrown. Refer to `InitializationTask.java`.

```

1. @Override
2. protected InitStatus doInBackground(final Void... params) {
3.     InitStatus status = InitStatus.NO_ERROR;
4.
5.     try {
6.         // initialize Workpath SDK
7.         Workpath.getInstance().initialize(mContextRef.get());
8.     } catch (SdkUnsupportedException sue) {
9.         Log.e(TAG, "SDK is not supported!", sue);
10.        mException = sue;
11.        status = InitStatus.INIT_EXCEPTION;
12.    } catch (SecurityException se) {
13.        Log.e(TAG, "Security exception!", se);
14.        mException = se;
15.        status = InitStatus.INIT_EXCEPTION;
16.    }
17.
18.    // Check if PrinterService is supported
19.    if (status == InitStatus.NO_ERROR
20.        && (!PrinterService.isSupported(mContextRef.get())
21.            || !JobService.isSupported(mContextRef.get()))) {
22.        // PrinterService is not supported on this device
23.        status = InitStatus.NOT_SUPPORTED;
24.    }
25.
26.    return status;
27. }

```

Exception handling by initializationTask

5 Integrating with print functions

The structure of Print Sample contains configuring print attributes and retrieving capabilities for a job.

5.1 Configuring Print Attributes

The print service API lets the app control various sets of print attributes. To receive user inputs for each of these print attributes, Print Sample uses the *PreferenceFragment* and allows the user to configure them manually. Please see [PrintConfigureFragment.java](#).

```

1.  /**
2.   * Simple {@link PreferenceFragmentCompat} to set Print Attributes and
3.   * save into preferences.
4.   */
5.  public final class PrintConfigureFragment extends PreferenceFragmentCompat implements S
   haredPreferences.OnSharedPreferenceChangeListener {
6.      // Preferences keys for PrintAttributes
7.      public static final String PREF_COPIES = "pref_copies";
8.      public static final String PREF_COLOR_MODE = "pref_colorMode";
9.      public static final String PREF_DUPLEX_MODE = "pref_duplexMode";
10.     public static final String PREF_FILENAME = "pref_filename";
11.     public static final String PREF_USB_STORAGE = "pref_usb_storage";
12.     public static final String PREF_USB_FILENAME = "pref_usb_filename";
13.     public static final String PREF_STREAM_FILENAME = "pref_stream_filename";
14.     public static final String PREF_AUTOFIT = "pref_autoFit";
15.     public static final String PREF_STAPLE_MODE = "pref_stapleMode";
16.     public static final String PREF_COLLATE_MODE = "pref_collateMode";
17.     public static final String PREF_PAPER_SOURCE = "pref_paperSource";
18.     public static final String PREF_PAPER_SIZE = "pref_paperSize";
19.     public static final String PREF_PAPER_TYPE = "pref_paperType";
20.     public static final String PREF_DOC_FORMAT = "pref_documentFormat";
21.     public static final String PREF_SOURCE = "pref_source";
22.     public static final String PREF_URI = "pref_uri";
23.     public static final String PREF_URI_USERNAME = "pref_uri_username";
24.     public static final String PREF_URI_PASSWORD = "pref_uri_password";

```

Configuring print attributes using preferenceFragment

PreferenceFragment contains methods to load the default values into the user interface, and to provide selection of the set values in accordance with each Print attribute's capability. The sample code shows how to use the list of capabilities and populate supported values for each attribute. It also sets default values for each attribute :

```

1.  /**
2.   * Fills preferences with received capabilities.
3.   *
4.   * @param caps {@link com.hp.workpath.api.printer.PrintAttributesCaps}
5.   */
6.  public void loadCapabilities(final PrintAttributesCaps caps) {
7.      final ArrayList<CharSequence> duplexEntries = new ArrayList<>();
8.      final ArrayList<CharSequence> duplexEntryValues = new ArrayList<>();
9.      // Load duplex
10.     for (Duplex duplex: caps.getDuplexList()) {
11.         duplexEntries.add(duplex.name());
12.         duplexEntryValues.add(duplex.name());
13.     }
14.     mDuplexPref.setEntries(duplexEntries.toArray(new CharSequence[duplexEntries.size()])
15.     ));
16.     mDuplexPref.setEntryValues(duplexEntryValues.toArray(new CharSequence[duplexEntryVa
17.     lues.size()]));
18.     mDuplexPref.setDefaultValue(Duplex.DEFAULT.name());
19.     mDuplexPref.setValueIndex(0);
20.     mDuplexPref.setSummary("%s");
21.     // Load color mode
22.     final ArrayList<CharSequence> cmEntries = new ArrayList<>();
23.     final ArrayList<CharSequence> cmEntryValues = new ArrayList<>();
24.     for (final ColorMode cm: caps.getColorModeList()) {
25.         cmEntries.add(cm.name());
26.         cmEntryValues.add(cm.name());
27.     }
28.     mCMPref.setEntries(cmEntries.toArray(new CharSequence[cmEntries.size()]));
29.     mCMPref.setEntryValues(cmEntryValues.toArray(new CharSequence[cmEntryValues.size()])
30.     ));
31.     mCMPref.setDefaultValue(ColorMode.DEFAULT.name());
32.     mCMPref.setValueIndex(0);
33.     mCMPref.setSummary("%s");
34.     // Load AutoFit
35.     final ArrayList<CharSequence> afEntries = new ArrayList<>();
36.     final ArrayList<CharSequence> afEntryValues = new ArrayList<>();
37.     for (final AutoFit af: caps.getAutoFitList()) {
38.         afEntries.add(af.name());
39.         afEntryValues.add(af.name());
40.     }
41.     mAFPref.setEntries(afEntries.toArray(new CharSequence[afEntries.size()]));
42.     mAFPref.setEntryValues(afEntryValues.toArray(new CharSequence[afEntryValues.size()])
43.     ));
44.     mAFPref.setDefaultValue(AutoFit.DEFAULT.name());
45.     mAFPref.setValueIndex(0);
46.     mAFPref.setSummary("%s");

```

Loading capabilities

NOTE: For the sake of simplicity, Samples use English language representations for Enums. You should use localized strings instead. Default values and capability values for each attribute are obtained from *PrinterService*. This is explained in the next section.

5.2 Retrieving Capabilities

5.2.1 Needs of Capabilities

The capability of print service attributes is retrieved to ensure that the user is selecting one of the allowed values. Since the capability of print attribute can vary among device's, it is very important to prevent the user from selecting unsupported attribute values as you develop your app.

5.2.2 Reading Capabilities

Reading capability requires interaction with a device and must not be invoked from the UI thread. Therefore, `MainActivity` executes `LoadCapabilitiesTask()`, which then calls the `requestCaps()` method obtaining values from `getCapabilities` of `PrinterService` to read the capabilities.

The `PrinterService.getCapabilities(context, result)` will return any errors that exist in `result` argument.

```
1. /**
2.  * Requests printer Print capabilities
3.  *
4.  * @param context {@link Context} to obtain data
5.  * @return {@link com.hp.workpath.api.printer.PrintAttributesCaps}
6.  */
7. public PrintAttributesCaps requestCaps(final Context context, Result result) {
8.     if (result == null) {
9.         result = new Result();
10.    }
11.
12.    // cache capabilities for building PrintAttributes
13.    mCapabilities = PrinterService.getCapabilities(context, result);
14.
15.    if (result.getCode() == Result.RESULT_OK && mCapabilities != null) {
16.        Log.i(TAG, "Caps=" + LogUtil.getPrintCapabilitiesToString(mCapabilities));
17.    } else {
18.        showResult("PrinterService.getCapabilities()", result);
19.    }
20.
21.    return mCapabilities;
22. }
```

Loading capabilities

5.2.3 Getting Printer Status and Tray information

The `PrinterService.getStatus(context, result)` returns printer status and `PrinterService.getTrays(context, result)` returns Tray information from a device.

```
1. private void getPrinterStatus() {
2.     if (PrinterStatus.isSupported(getContext())) {
3.         Result result = new Result();
4.         StatusInfo statusInfo = PrinterStatus.getStatus(getContext(), result);
5.         if (result.getCode() == Result.RESULT_OK) {
6.             Log.i(TAG, "StatusInfo=" + LogUtil.getStatusInfoToString(statusInfo, true))
7.         ;
8.             mPrinterStatus = LogUtil.getStatusInfoToString(statusInfo, false);
9.             mStatusTextView.setText(mPrinterStatus);
10.        } else {
11.            showResult("PrinterStatus.getStatus()", result);
12.        }
13.    } else {
14.        showResult("PrinterStatus is not supported", null);
15.    }
```

Getting PrinterStatus information in StatusPreference.java

```

1. private void getTrayInfoStatus() {
2.     if (PrinterStatus.isSupported(getContext())) {
3.         Result result = new Result();
4.         List<TrayInfo> trayInfoList = PrinterStatus.getTrays(getContext(), result);
5.         if (result.getCode() == Result.RESULT_OK) {
6.             mTrayInfoStatus1 = "";
7.             mTrayInfoStatus2 = "";
8.             int index = 0;
9.             for (TrayInfo trayInfo : trayInfoList) {
10.                Log.i(TAG, "trayInfo=" + LogUtil.getTrayInfoToString(trayInfo, true));
11.                String status;
12.                if (trayInfo.getStatus() == TrayInfo.Status.AVAILABLE) {
13.                    status = trayInfo.getPaperSource() + " (" + trayInfo.getStatus() +
14.                        + trayInfo.getLevel() + "% cap:" + trayInfo.getCapacity() +
15.                        + trayInfo.getPaperSize() + ", " + trayInfo.getPaperType()
16.                    } else {
17.                        status = trayInfo.getPaperSource() + " (" + trayInfo.getStatus() +
18.                    }
19.                    if (index < 3) {
20.                        mTrayInfoStatus1 += status;
21.                    } else {
22.                        mTrayInfoStatus2 += status;
23.                    }
24.                    index++;
25.                }
26.                mTrayInfoTextView1.setText(mTrayInfoStatus1);
27.                mTrayInfoTextView2.setText(mTrayInfoStatus2);
28.            } else {
29.                showResult("PrinterStatus.getTrays()", result);
30.            }
31.        } else {
32.            showResult("PrinterStatus is not supported", null);
33.        }
34.    }

```

Getting Tray information in TrayInfoPreference.java

5.2.4 Getting defaults

The `PrinterService.getDefault(context, result)` returns default attributes of print from a device.

```
1. @Override
2. protected PrintAttributes doInBackground(final Void... params) {
3.     return PrinterService.getDefault(mContextRef.get(), result);
4. }
5.
6. @Override
7. protected void onPostExecute(final PrintAttributes defaults) {
8.     super.onPostExecute(defaults);
9.
10.    if (defaults != null && result.getCode() == Result.RESULT_OK) {
11.        Log.i(TAG, "Defaults=" + LogUtil.getPrintAttributesToString(defaults));
12.        mContextRef.get().showResult(mContextRef.get().getString(R.string.succeed));
13.        mFragment.get().setDefaultPrintAttributes(defaults);
14.    } else {
15.        mContextRef.get().showResult("PrinterService.getDefault()", result);
16.    }
17. }
```

Getting defaults

5.2.5 Error handling

PrinterService can return a null capability list instead of providing the Print Attribute Capability. In this case, the application will not be able to load the values into the user interface, and it would result in exceptions when a job is submitted. Therefore, be sure to take care of null responses from **PrinterService**.

5.3 Print job submission

Print job submission involves:

- Creating job attributes with Builders
 - `PrintFromStorageBuilder`
 - `PrintFromHttpBuilder`
 - `PrintFromUsbBuilder`
 - `PrintFromStreamBuilder`
- Submitting the print job
- Observing the job events and starting monitoring of job state.

5.3.1 Batch Jobs

This feature is implemented to support batch printing. Before WorkpathLib 1.6.1, it only supported single `PrintAttributes` object. So, if an user has multiple jobs to process, the user need to run them step by step.

After WorkpathLib 1.6.1, it will support multiple `PrintAttributes` at once so a user can process multiple jobs at once. STREAM source mode does not support batch job operations.

Below snippet shows how to make multiple `PrintAttributes` objects and add it on a `ArrayList<PrintAttributes>` object to call submit API for batch jobs.

NOTE: When more than one batch is submitted (e.g., batch1, batch2, batch3), the batches will be queued and processed in FIFO (First-In-First-Out) order. This means that batch1 will be processed first, followed by batch2, and then batch3. The next batch will start processing only after the previous batch has completed.

```
1. ArrayList<PrintAttributes> batchJobs = new ArrayList<>();
2. PrintAttributes attributes1 = new PrintAttributes.PrintFromStorageBuilder()...
3. batchJobs.add(attributes1)
4. PrintAttributes attributes2 = new PrintAttributes.PrintFromStorageBuilder()...
5. batchJobs.add(attributes2)
6. PrintAttributes attributes3 = new PrintAttributes.PrintFromStorageBuilder()...
7. batchJobs.add(attributes3)
8. ...
9. PrinterService.submit(mContextRef.get(), batchJobs, taskAttribs);
```

5.3.2 Creating Job Attributes for Print

`PrintAttributes` class provides flexibility in terms of the type of builder used to create print attributes. The simple method for printing a file from the `localstorage(default is {app-private folder}/files)` is called with `PrintFromStorageBuilder`, and for printing a file from http url is called with `PrintFromHttpBuilder`.

From SDK 1.1, `PrintFromUsbBuilder` for printing a file from USB and `PrintFromStreamBuilder` for printing bytes stream have been supported.

1. PrintFromStorageBuilder

“PrintFromStorageBuilder” is for printing a document from local storage on a device.

```
1. if (source == PrintAttributes.Source.STORAGE) {
2.     String filePath = mPrefs.getString(PrintConfigureFragment.PREF_FILENAME, "");
3.     Log.i(TAG, "Selected path: " + filePath);
4.
5.     attributes =
6.         new PrintAttributes.PrintFromStorageBuilder(Uri.fromFile(new File(filePath)
7.     ))
8.         .setCollateMode(collateMode)
9.         .setColorMode(cm)
10.        .setDuplex(duplex)
11.        .setAutoFit(af)
12.        .setStapleMode(sm)
13.        .setPaperSource(psrc)
14.        .setPaperSize(psz)
15.        .setPaperType(paperType)
16.        .setDocumentFormat(dfmt)
17.        .setCopies(copies)
18.        .setJobName(jobName)
19.        .build(caps);
```

Building attributes with PrintFromStorageBuilder

2. PrintFromHttpBuilder

“PrintFromHttpBuilder” is for printing a document from http url. Valid http url should be built with attributes.

```

1. } else if (source == PrintAttributes.Source.HTTP) {
2.     String fileUri = mPrefs.getString(PrintConfigureFragment.PREF_URI, "");
3.     Log.i(TAG, "Selected uri: " + fileUri);
4.
5.     String fileUriUsername = mPrefs.getString(PrintConfigureFragment.PREF_URI_USERNAME,
6.     "");
7.     String fileUriPassword = mPrefs.getString(PrintConfigureFragment.PREF_URI_PASSWORD,
8.     "");
9.
10.    NetworkCredentialsAttributes networkCredentialsAttributes = null;
11.    if (!TextUtils.isEmpty(fileUriUsername) && !TextUtils.isEmpty(fileUriPassword)) {
12.        networkCredentialsAttributes = new NetworkCredentialsAttributes.Builder()
13.            .setUserName(fileUriUsername)
14.            .setPassword(fileUriPassword)
15.            .build();
16.    }
17.
18.    // building with common print attributes set
19.    attributes = new PrintAttributes.PrintFromHttpBuilder(Uri.parse(fileUri))
20.        .setCollateMode(collateMode)
21.        .setColorMode(cm)
22.        .setDuplex(duplex)
23.        .setAutoFit(af)
24.        .setStapleMode(sm)
25.        .setPaperSource(psrc)
26.        .setPaperSize(psz)
27.        .setPaperType(paperType)
28.        .setDocumentFormat(dfmt)
29.        .setCopies(copies)
30.        .setJobName(jobName)
31.        .setNetworkCredentials(networkCredentialsAttributes)
32.        .build(caps);

```

Building attributes with PrintFromHttpBuilder

3. PrintFromUsbBuilder

“PrintFromUsbBuilder” is for printing a document from usb. Storage path should be retrieved using `MassStorageService`. The code shows how to call api.

```

1. List<MassStorageInfo> storageList = MassStorageService.getStorageList(getActivity(), re
   sult);
2. if (result.getCode() != Result.RESULT_OK || storageList == null) {
3.     storageList = Collections.emptyList();
4. }
5.
6. final List<CharSequence> entries = new ArrayList<>();
7. final List<CharSequence> entriesValues = new ArrayList<>();
8.
9. for (MassStorageInfo storage : storageList) {
10.    if (storage.getType() == MassStorageInfo.StorageType.USB && storage.isMounted()) {
11.        entries.add(storage.getName());
12.        entriesValues.add(storage.getExternalFileDirectory());
13.    }
14. }

```

Retrieving storage path using MassStorageService api

```

1. } else if (source == PrintAttributes.Source.USB) {
2.     String filePath = mPrefs.getString(PrintConfigureFragment.PREF_USB_FILENAME, "");
3.
4.     Log.i(TAG, "Selected usb file: " + filePath);
5.
6.     // building with common print attributes set
7.     attributes = new PrintAttributes.PrintFromUsbBuilder(filePath)
8.         .setCollateMode(collateMode)
9.         .setColorMode(cm)
10.        .setDuplex(duplex)
11.        .setAutoFit(af)
12.        .setStapleMode(sm)
13.        .setPaperSource(psrc)
14.        .setPaperSize(psz)
15.        .setDocumentFormat(dfmt)
16.        .setCopies(copies)
17.        .setJobName(jobName)
18.        .build(caps);

```

Building attributes with PrintFromUsbBuilder

4. PrintFromStreamBuilder

“PrintFromStreamBuilder” is for printing a file as stream type. This builder has a dependency of supported files per device capability. For examples, image file is not accepted in this builder.

```
1. } else if (source == PrintAttributes.Source.STREAM) {
2.     String filePath = mPrefs.getString(PrintConfigureFragment.PREF_STREAM_FILENAME, "")
3.     ;
4.     Log.i(TAG, "Selected file for stream: " + filePath);
5.
6.     // can print from any InputStream, here FileInputStream as example
7.     InputStream printStream = new FileInputStream(new File(filePath));
8.
9.     // building with common print attributes set
10.    attributes = new PrintAttributes.PrintFromStreamBuilder(printStream)
11.        .setCollateMode(collateMode)
12.        .setColorMode(cm)
13.        .setDuplex(duplex)
14.        .setAutoFit(af)
15.        .setStapleMode(sm)
16.        .setPaperSource(psrc)
17.        .setPaperSize(psz)
18.        .setDocumentFormat(dfmt)
19.        .setCopies(copies)
20.        .setJobName(jobName)
21.        .build(caps);
22. }
```

Building attributes with PrintFromStreamBuilder

NOTE: As `CapabilitiesExceededException` does not provide a method to determine which attribute was the trigger, it is not possible to flag a given field. It recommends that you validate values before building print attributes.

5.3.3 Background/ Foreground Jobs

This feature is implemented to support multiple jobs at a time. Before WorkpathLib 1.6.1, only foreground job was supported. The limitation with this was that when a new job is submitted while another job is under process, then the new job will fail.

After WorkpathLib 1.6.1, if you set and submit a job with a background job property then it will wait until the previous job is finished. STREAM source mode only supports foreground jobs.

Background job parameter can be set by the method `setBackgroundJob` as shown below:

```
1. final PrintletAttributes taskAttribs = new PrintletAttributes.Builder()
2.     .setShowSettingsUi(settingsUi)
3.     .setBackgroundJob(true) // true : background job // false : foreground job - by default
4.     .build();
5.
6. ...
7. PrinterService.submit(mContextRef.get(), attributes, taskAttribs);
```

5.3.4 Submitting a job

Sample shows how an application creates print job and checks job status. Main method for submitting a job is `PrinterService.submit()`. This method allows to put `PrintletAttributes` as a parameter to show embedded settings dialog.

The following code shows how to implement it with `PrintletAttributes` and `PrintAttributes`.

```
1. final PrintletAttributes taskAttribs = new PrintletAttributes.Builder()
2.     .setShowSettingsUi(settingsUi)
3.     .setBackgroundJob(isBackGround)
4.     .build();
5.
6. // Submit the job
7. if (ButtonClicked == PRINT_CLICKED && batchJobs.isEmpty()) {
8.     return PrinterService.submit(mContextRef.get(), attributes, taskAttribs);
9. } else if (ButtonClicked == PRINT_CLICKED && !batchJobs.isEmpty()) {
10.    return PrinterService.submit(mContextRef.get(), batchJobs, taskAttribs);
11. }
```

Submitting a print job with attributes

5.4 Monitoring Job

When a job is submitted, the app should be aware of the status of the request, such as whether it is progressing, completed, canceled or failed. Thus, Print Sample shows how to register an observer to check the status of the submitted job.

Application should extend `JobService.AbstractJobletObserver` for overriding `onComplete`, `onCancel`, `onProgress` and `onFail` events. It acts as observer of the job state and receives job events.

Canceling or completed jobs display a toast to the user as an indication. If the job fails, sample notifies the user of the failure with a status code and detailed logs, as shown below.

```

1.  /**
2.   * Observer for submitted job
3.   */
4.  private class JobObserver extends JobService.AbstractJobletObserver {
5.
6.      public JobObserver(final Handler handler) {
7.          super(handler);
8.      }
9.
10.     public void onProgress(final String rid, final JobInfo jobInfo) {
11.         Log.i(TAG, "onProgress: Received rid=" + rid);
12.         Log.i(TAG, "JobInfo=" + LogUtil.getJobInfoToString(jobInfo));
13.         if (rid.equals(mRid)) {
14.             if (mJobId == null) {
15.                 if (jobInfo.getJobId() != null) {
16.                     mJobId = jobInfo.getJobId();
17.
18.                     Log.i(TAG, "Received jobId=" + mJobId);
19.                     showSnackBar(getString(R.string.job_id, mJobId));
20.
21.                     if (mPrefs.getBoolean(PrintConfigureFragment.PREF_MONITORING_JOB, false)) {
22.                         // Store Job Id in order to verify it in the Broadcast Receiver
23.
24.                         mPrefs.edit().putString(PrintConfigureFragment.CURRENT_JOB_ID,
25. mJobId).apply();
26.
27.                         final Intent intent = new Intent(getApplicationContext(), JobCompleteReceiver.class);
28.                         intent.setAction(ACTION_PRINT_COMPLETED);
29.                         intent.putExtra(JobCompleteReceiver.RID_EXTRA, rid);
30.                         intent.putExtra(JobCompleteReceiver.JOB_ID_EXTRA, mJobId);
31.
32.                         final boolean showProgress =
33.                             mPrefs.getBoolean(PrintConfigureFragment.PREF_SHOW_JOB_PROGRESS, true);
34.
35.                         // Monitor the job completion
36.                         final JobletAttributes taskAttributes =
37.                             new JobletAttributes.Builder().setShowUi(showProgress).
38. build();
39.
40.                         final String jrid = JobService.monitorJobInForeground(MainActivity.this, mJobId,
41. taskAttributes, intent);

```

```

39.
40.         Log.i(TAG, "MonitorJob request: " + jrid);
41.     }
42. }
43.
44. }
45. }
46.
47. @Override
48. public void onComplete(final String rid, final JobInfo jobInfo) {
49.     Log.i(TAG, "onComplete: Received rid=" + rid);
50.     Log.i(TAG, "JobInfo=" + LogUtil.getJobInfoToString(jobInfo));
51.     if (jobInfo.getJobType() == JobInfo.JobType.PRINT) {
52.         showSnackBar(getString(R.string.job_completed, jobInfo.getJobName()));
53.     }
54. }
55.
56. @Override
57. public void onFail(final String rid, final Result result) {
58.     Log.e(TAG, "onFail: Received rid=" + rid);
59.     showResult(getString(R.string.job_failed), result);
60. }
61.
62. @Override
63. public void onCancel(final String rid) {
64.     Log.i(TAG, "onCancel: Received rid=" + rid);
65.     showSnackBar(getString(R.string.job_cancelled));
66. }
67. }

```

Receiving events of a print job state

NOTE: `JobService.monitorJobInForeground()` provides a progress dialog for showing job state by Workpath SDK. If application doesn't have custom dialog, this api will help to provide feature easily. Sample also explains how to call the method.

6 Legal notice

The information contained in this documentation is subject to change without notice.

HP Inc. makes no warranty of any kind with regard to this material, including, but not limited to, the implied warranties of merchantability and fitness for a particular purpose. HP Inc. shall not be liable for errors contained herein or for incidental or consequential damages in connection with the furnishing, performance, or use of this material.

This document contains proprietary information that is protected by copyright. All rights are reserved. No part of this documentation may be transformed, reproduced, or translated to another language without prior.

© Copyright 2018 HP Development Company, L.P.

7 Support

The HP Solution Business Program (SBP) support staff has established a procedure for submitting technical problems to the Incident Call Center (ICC). All SBP members must be registered with the Solution Programs Portal (SPP) to access technical support and other feedback or reporting activities.

Log on to the portal at <<http://www.hp.com/go/solutions>> to submit your issue to technical support. After you submit the issue, a report will be generated for the Incident Call Center. A Technical Support representative will be assigned to validate the incident and to work on a fix. HP will contact you for further information regarding the incident as quickly as possible.